

PROBLEM

Predicting Customer Churn at Infrastructure Scale

- **Business problem:**

telecom churn costs 5-25x more to replace than retain; predict the 14% of customers who will leave from usage behavior.

- **Dataset:**

Churn_Dataset.csv - 5,000 customers, 16 numeric features, 341 KB. Model: MLP 128-64-32 (JAX/Flax + PyTorch), 92-94% test accuracy vs 86% majority baseline.

- **Resource challenge:**

the dataset is tiny but the pipeline must be production-shaped: every hardware tier is oversized, so FIXED costs (compile, kernel dispatch, staging) dominate.

- **Question:**

which rung of the hardware ladder (1 CPU core → 10 cores → GPU → cluster GPU/TPU) does this workload actually need, and what is the real bottleneck?

- **Constraint:**

Stanford SLURM cluster (hpcc-cluster-49) is VPN-gated and was unreachable; cluster rungs are provisioned as manifests, local rungs are measured.

Immutable Containers, Declarative Orchestration, XLA

- **Environment (Docker):**

two pinned images - python:3.12-slim CPU and CUDA 12.4+cuDNN - all deps baked at build; nothing installed on nodes; Apptainer .sif on SLURM.

- **Orchestration (K8s + SLURM):**

GKE Jobs with exact requests/limits (8 CPU / 24 GiB / 1×L4; TPU v5e 2×2 = 4 chips) · sbatch scripts with --cpus-per-task/--mem/--gres · 2-node jax.distributed.

- **Storage tier:**

NFS / GCS canonical copy → staged once to node-local NVMe scratch (\$SLURM_TMPDIR, local-SSD emptyDir, gcsfuse cache) → RAM → device memory. Training loop never touches shared storage.

- **Compilation (JAX/XLA):**

entire train step (fwd + bwd + Adam) jit-fused into one XLA computation; torch.compile (Inductor) wired for the accelerator path; bf16 flag for A100/TPU rungs.

MEASUREMENTS

Telemetry: Honest Step Times + Node Metrics

- **Scaling matrix:**

3 discrete scale steps - single-core CPU (JAX/XLA), full 10-core node (JAX/XLA), Apple M4 GPU (PyTorch/MPS) - plus batch sweeps and a bf16 ablation: 12 configs × 400 epochs.

- **Step timing:**

per-step wall clock with forced device sync (`jax.block_until_ready` / `torch.mps.synchronize`); first traced+compiled step timed separately = compile overhead.

- **Node telemetry:**

background psutil sampler at 5 Hz: whole-node CPU %, process RSS; GPU driver memory via `torch.mps`; `nvidia-smi` 1 Hz hook baked into the SLURM GPU job.

- **IO metrics:**

ingestion tier is staged + device-resident by design → input pipeline contributes zero steady-state stalls (verified: one 341 KB read per job).

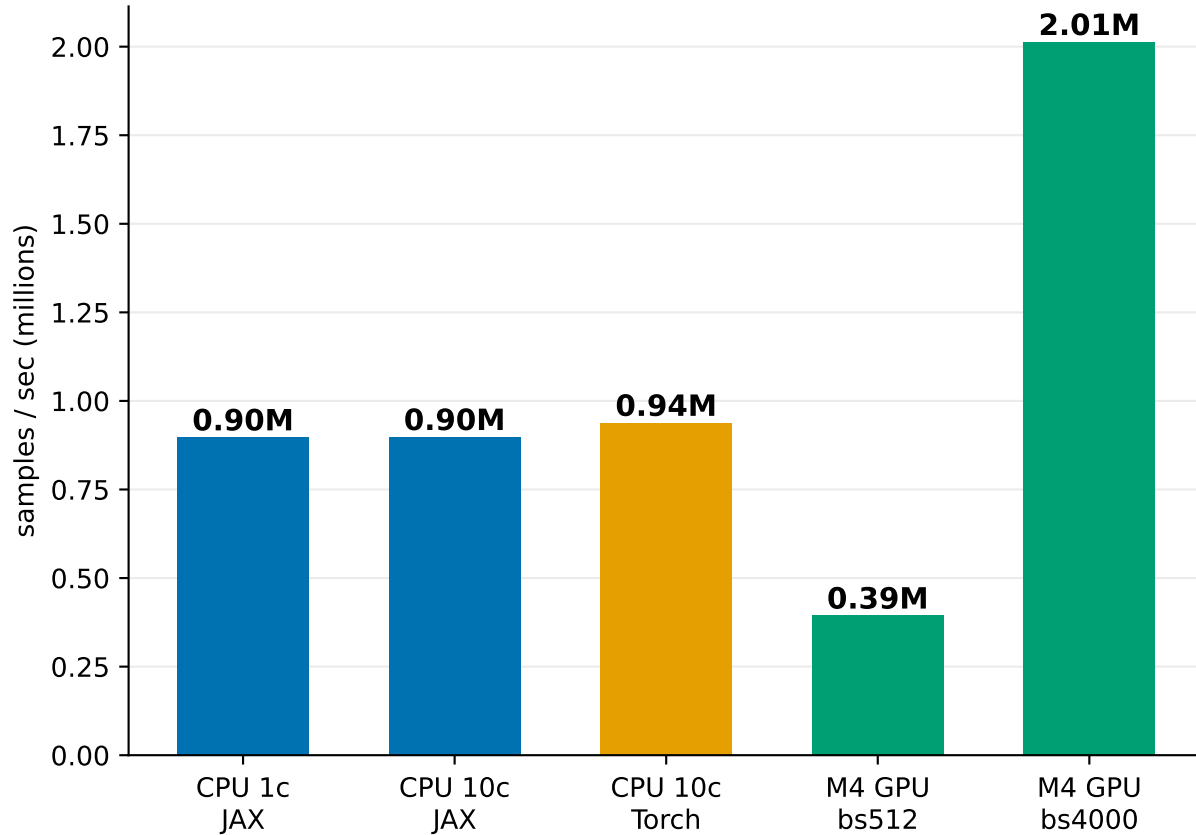
- **Output:**

every run emits one JSON record → pandas matrix → figures → this deck. Fully scripted: `benchmarks/run_matrix.sh`.

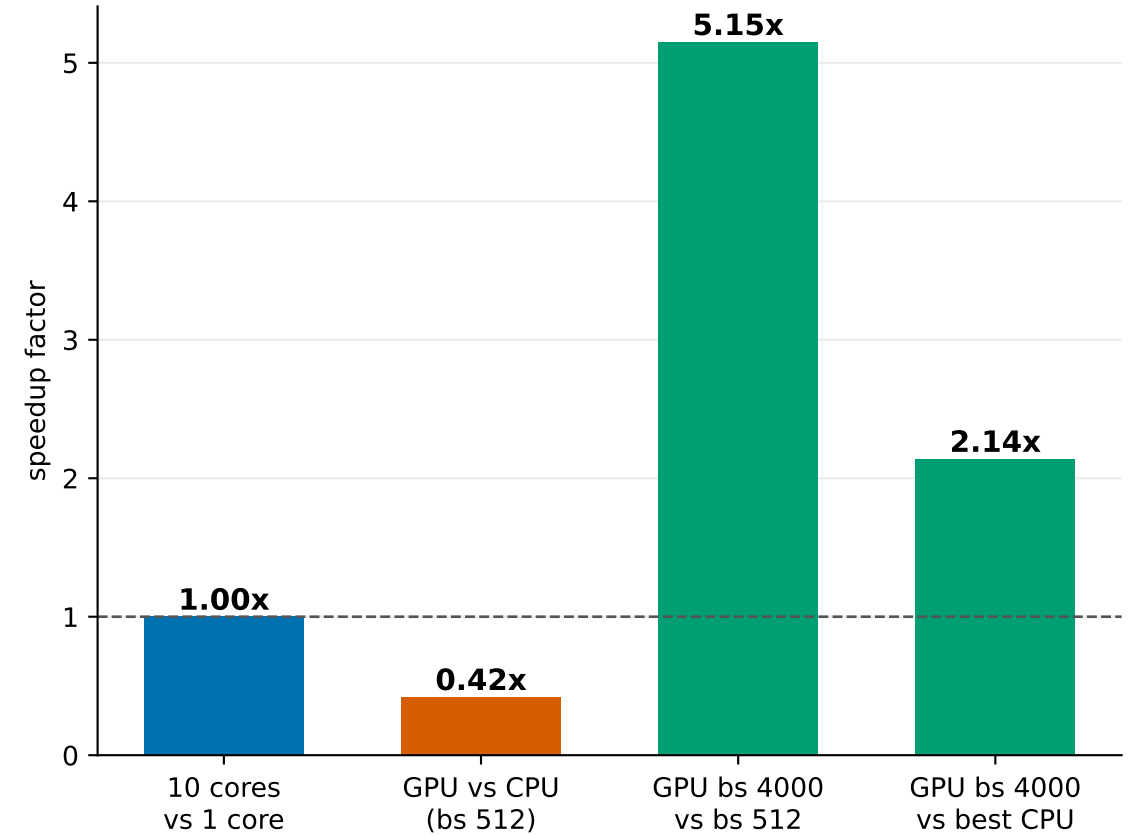
RESULTS - HARDWARE COMPARISON

10 Cores Buy Nothing; Batch Size Buys 5.2x

Training throughput (400 epochs)



Speedup factors (dashed = parity)



Utilization never saturated: node CPU $\leq 25\%$, GPU memory 19 MB, peak RSS 0.44 GB / 16 GB.
Compile overhead 0.16 s (JAX) vs 0.01 s (eager torch). bf16 on M4 CPU: 0.75x, a measured regression.
churn-hpc · Anh Quang Nguyen · July 2026

CONCLUSION

The Bottleneck Is Dispatch Latency, Not Hardware

■ Bottleneck identified:

fixed per-step host-side launch cost. Evidence: core-count indifference (1.00x), GPU step 3x CPU step for identical math, throughput ~linear in batch size, no resource saturated.

■ What worked:

batch 512→4000 amortized launch cost for 5.15x; XLA fusion gave 0.4 ms CPU steps; device-resident data removed IO entirely. What didn't: bf16 on CPU (0.75x), more cores (1.00x).

■ Cost trade-off:

GPU saves ~0.8 s per run but costs ~4x more per hour than a CPU node - accelerators earn their premium here only for parallel sweeps, not for shrinking one small job.

■ Scaling recommendation:

< ~1M rows: one CPU node with XLA. Grow batch (with LR warmup) before growing hardware; fuse epochs with `jax.lax.scan` next. GPU/TPU + multinode manifests are provisioned for the real rung: cluster-scale sweeps once VPN access returns.

■ Lesson:

measure fixed costs before buying parallelism - the scaling matrix falsified two 'obvious' upgrades (more cores, an accelerator at default batch) in under 30 s of compute.